LinkML proof of concept: id and subject (Individual)

Summary of work

2026-11-05

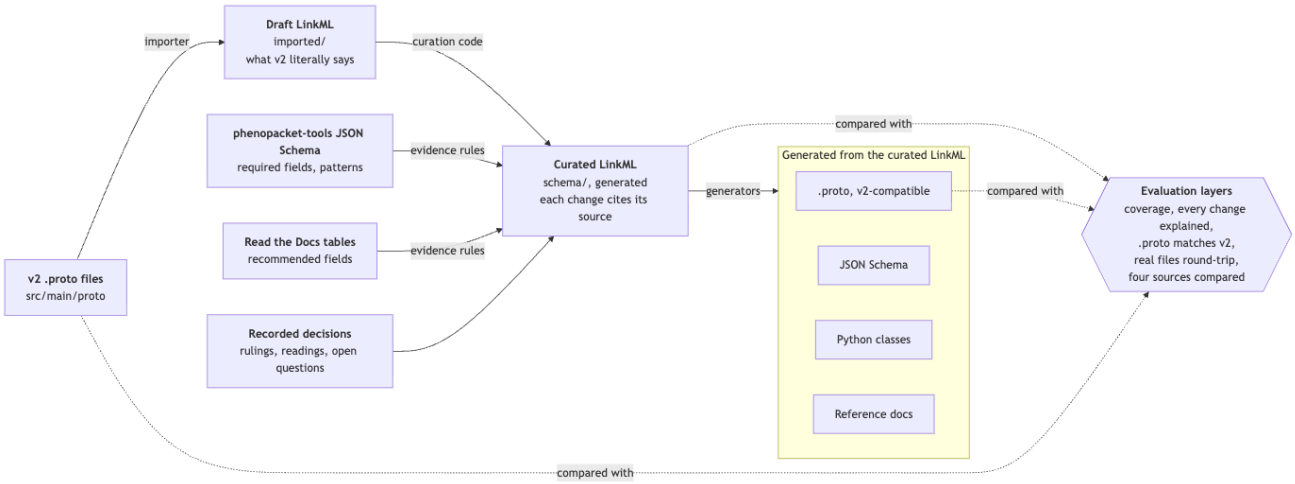
The answer

Yes. One LinkML definition of `Phenopacket.id`, `Phenopacket.subject` and Individual generates:

- a `.proto` with **0 differences** from v2 (across 87 elements), with all 60 of v2's comments carried over word for word;
- a JSON Schema that checks the current docs' own examples (3 of 3 as expected);
- reference docs in the current Read the Docs style.

Every change from v2 cites its source. Everything is in the `linkml/` folder of the `feat/schema-linkml` branch.

How it works



1. **Import.** A script turns v2's `.proto` files into draft LinkML, keeping every name, number and comment.
2. **Curate.** Curation code turns the drafts into the source of truth: evidence rules read phenopacket-tools' JSON Schema (required fields, patterns) and the Read the Docs tables (recommended fields), and recorded decisions cover the rest. Each change cites its source; nothing is edited by hand.
3. **Generate** the `.proto`, JSON Schema and docs from the LinkML.
4. **Check** them: the `.proto` must match v2, and the docs' own examples must validate as expected.

The LinkML for `id`

v2's protos can't mark a field as required, so curation adds the rule:

```yaml
Phenopacket:
  attributes:
    id:
      rank: 1
      range: string
```

```
    description: An identifier specific for this phenopacket.
    annotations:
      proto_ref: org.phenopackets.schema.v2.Phenopacket.id
    required: true
    pattern: .
```

The changes curation made, and why:

- `pattern: .`, by the `not-blank` rule. A required string can't be blank: protobuf drops an empty string when writing, so blank and missing are the same on the wire.
- `required: true`, by the `required-from-tools` rule. Required, following the phenopacket-tools JSON Schema. Source: phenopacket-schema.json:7.

For these fields, this is what each source says:

Field	v2 proto comment	This LinkML	phenopacket-tools	Docs table
Individual.id	no marker	required	required	required
Phenopacket.id	no marker	required	required	required
Phenopacket.subject	no marker	optional	optional	optional

The generated .proto

The generated Individual has the same definitions and comments as v2's (shown here without comments). LinkML's stock generator, by comparison, renames fields to their JSON names and loses the timestamp type:

```
message Individual {
  string id = 1;
  repeated string alternate_ids = 2;
  google.protobuf.Timestamp date_of_birth = 3;
  TimeElement time_at_last_encounter = 4;
  VitalStatus vital_status = 5;
  Sex sex = 6;
  KaryotypicSex karyotypic_sex = 7;
  OntologyClass gender = 8;
  OntologyClass taxonomy = 9;
}
```

```
message Individual {
  string id = 1;
  repeated string alternateIds = 2;
  string dateOfBirth = 3;
  TimeElement timeAtLastEncounter = 4;
  VitalStatus vitalStatus = 5;
  Sex sex = 6;
  KaryotypicSex karyotypicSex = 7;
  OntologyClass gender = 8;
  OntologyClass taxonomy = 9;
}
```

The generated docs

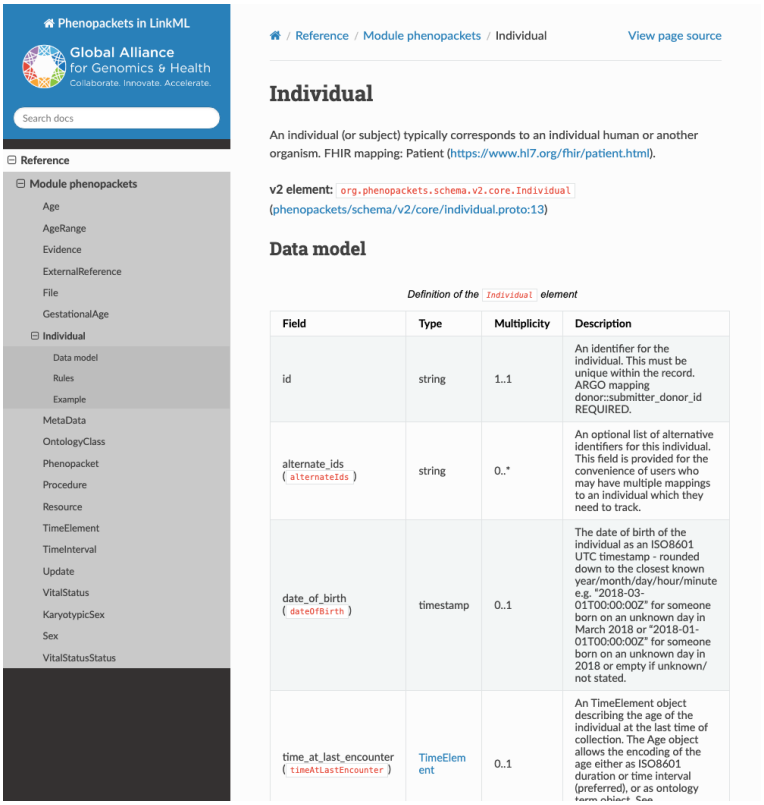


Figure 1: The generated Individual page, in the same theme and layout as today's site.

Test case: the docs' own examples

The YAML examples on the current Individual page, validated as written:

Example	Read as	Expected	Result	Missing recommended fields
Lines 67-70 (`individual`)	Individual	valid	valid	timeAtLastEncounter, vitalStatus
Lines 117-120 (`individual`)	Individual	valid	valid	timeAtLastEncounter, vitalStatus
Lines 126-132 (`htsFile`)	File	PXF-CLOSED-FILE	PXF-CLOSED-FILE	individualToFileIdentifiers

- **Both Individual examples are valid,** but each leaves out the two fields the page marks RECOMMENDED.
- **The file example still uses v1:** its key (`htsFile`) and four of its fields don't exist in v2.

Beyond Individual

The same approach was run on all of v2: 434 elements, 195 recorded changes, and a generated `.proto` with 0 differences from v2. LinkML's stock generator would differ in 433 places; a 151-line generator closes that gap. That work also found problems that exist today:

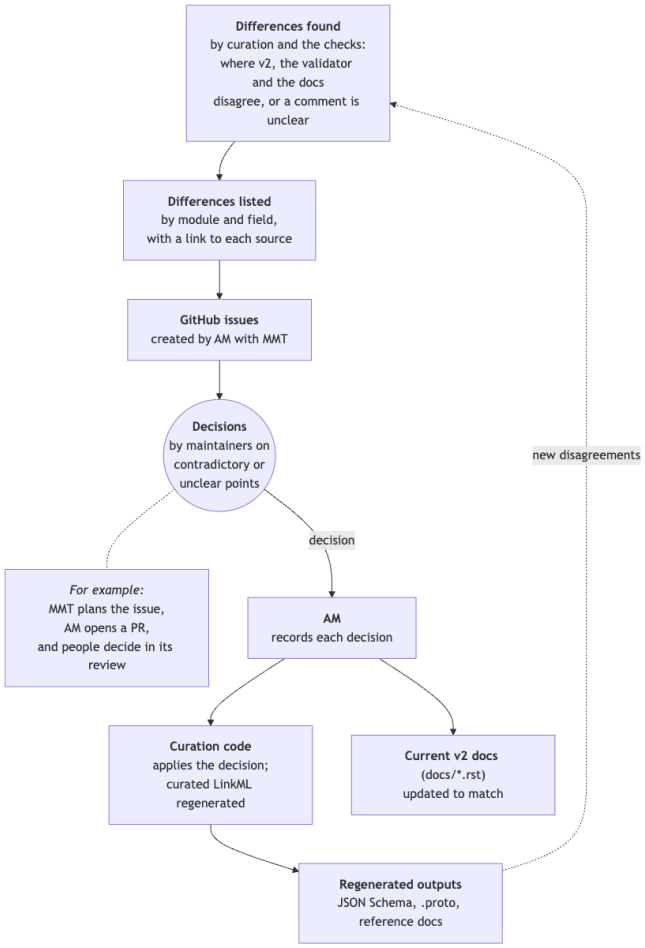
- phenopacket-tools rejects some valid v2 data, such as VRS objects with an id, and a zero in required numbers;

- the docs, the validator and v2's comments disagree in places.

Next steps: maintainer review

This curation is a first pass. Wherever the protobuf, the validator and the docs disagree, or a comment is unclear, the LinkML has to pick one answer, and maintainers should decide which. `differences.md` in the folder lists 7 examples, such as:

- The docs say a resource without a release version should leave `version` blank; the validator rejects a blank version.
- v2 marks only `OntologyClass.id` as REQUIRED; the validator also requires `label`.
- The docs mark two Individual fields RECOMMENDED; protobuf can't express that, and the validator doesn't check it.



Proposed:

1. Curation and the checks find where the sources disagree, or a comment is unclear.
2. The differences are listed by module and field, with a link to each source.
3. AM works with MMT to create a GitHub issue for each.
4. Maintainers decide on the contradictory or unclear points. How varies; for example, MMT plans the issue, AM opens a PR, and people decide in its review.
5. AM records each decision; the curation code applies it to the LinkML, which regenerates the JSON Schema, `.proto` and reference docs. AM also updates the current v2 docs to match.